



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس مبانی رایانش توزیع شده
تمرین دوم (۲)

استاد درس

محمدرضا شکورنیا

دستیار ارشد

پویا جمشیدی

دستیاران مسئول تمرین

کاظم عیارزاده

محمد افضل زاده

بهار ۱۴۰۵

فهرست مطالب

۳	۱ موضوع تمرین
۳	۲ هدف تمرین
۳	۳ قوانین کلی
۴	۴ ساختار پیشنهادی پروژه
۵	۵ نمای کلی سیستم
۶	۶ بخش اول: آشنایی با RPC و فناوری های مختلف آن
۹	۷ بخش دوم: پیاده سازی سیستم توزیع شده چند ماشین مجازی
۱۳	۸ بخش سوم: افزودن Pub/Sub و اعلان مصرف حافظه
۱۷	۹ مستندات و گزارش
۱۸	۱۰ موارد تحویلی
۱۹	۱۱ معیار ارزیابی
۲۰	۱۲ موارد کسر نمره
۲۱	۱۳ دستاوردهای تمرین

۱ موضوع تمرین

موضوع این تمرین آشنایی با مفاهیم *Remote Procedure Call*، طراحی سرویس های توزیع شده، ارتباط بین چند ماشین مجازی، جداسازی وظایف بین سرویس ها، و استفاده از الگوی *Publish/Subscribe* برای پایش و اعلان رخدادها است.

در این تمرین باید یک سیستم توزیع شده کوچک طراحی و پیاده سازی کنید که شامل چند ماشین مجازی مستقل باشد. هر ماشین مجازی نقش مشخصی در سیستم دارد و ارتباط بین اجزا باید از طریق شبکه انجام شود.

۲ هدف تمرین

اهداف اصلی این تمرین عبارت اند از:

- آشنایی نظری و عملی با مفهوم *RPC*
- شناخت تفاوت *RPC* با *REST* و ارتباطات مستقیم شبکه ای
- طراحی یک سیستم چند سرویسی روی چند ماشین مجازی
- پیاده سازی احراز هویت کاربران با استفاده از *RPC*
- جداسازی سرویس وب، سرویس احراز هویت، و سرویس فایل
- پیاده سازی یک سازوکار ساده *Publish/Subscribe*
- پایش مصرف حافظه یک سرویس و ارسال هشدار در صورت عبور از آستانه مشخص

۳ قوانین کلی

- زبان برنامه نویسی پیشنهادی برای پیاده سازی سرویس ها *Go* است.
- استفاده از زبان دیگر فقط با تایید تیم درس مجاز است.

- پروژه باید روی محیط لینوکسی اجرا شود.
- حداقل سه ماشین مجازی مستقل باید استفاده شود.
- سرویس ها باید از طریق آدرس شبکه ماشین های مجازی با هم ارتباط داشته باشند.
- اجرای همه اجزا روی یک ماشین یا فقط با *localhost* قابل قبول نیست.
- هر سرویس باید *README* مستقل برای اجرا و تست داشته باشد.
- تمام دستورات اجرا باید دقیق، قابل بازتولید، و بدون ابهام باشند.

۴ ساختار پیشنهادی پروژه

ساختار تحویل پروژه می تواند به شکل زیر باشد:

```
HW2/  
  
  report.pdf  
  
  rpc-study/  
  
    rpc-summary.pdf  
  
  web-vm/  
  
    main.go  
  
    templates/  
  
    static/  
  
    README.md  
  
  auth-vm/  
  
    main.go  
  
    users.json  
  
    README.md  
  
  file-vm/
```

```
main.go
files/
images/
README.md
pubsub/
  publisher.go
  subscriber.go
  README.md
screenshots/
```

۵ نمای کلی سیستم

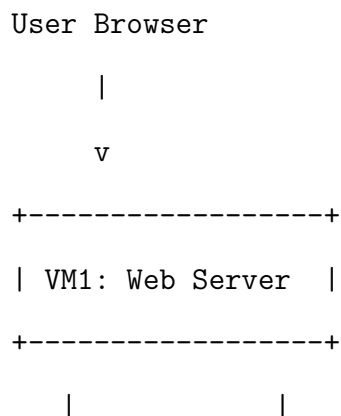
در این تمرین باید حداقل سه ماشین مجازی داشته باشید:

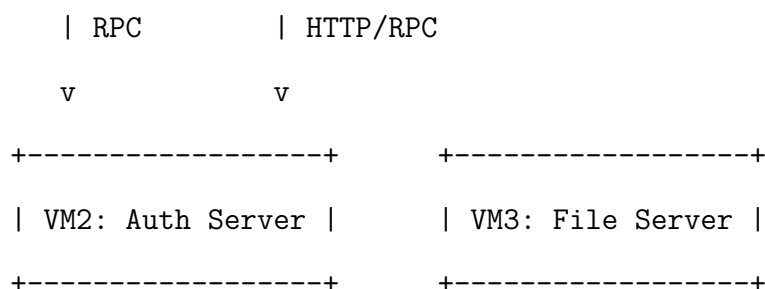
• *VM1*: سرویس وب و رابط کاربری

• *VM2*: سرویس احراز هویت کاربران

• *VM3*: سرویس فایل و تصاویر

ارتباط کلی سیستم باید به صورت زیر باشد:





Additionally:

VM1 publishes memory usage events.

A subscriber receives alerts when memory usage passes a threshold.

۶ بخش اول: آشنایی با RPC و فناوری های مختلف آن

۱-۰-۶ شرح بخش

در این بخش باید یک گزارش کامل درباره *RPC* تهیه کنید. این بخش می تواند صرفاً تئوری باشد، اما باید دقیق، فنی، و همراه با مقایسه فناوری های مختلف باشد.

۱-۶ موارد الزامی در گزارش

گزارش این بخش باید حداقل شامل موارد زیر باشد:

۱. تعریف *RPC*

۲. ایده اصلی فراخوانی تابع از راه دور

۳. تفاوت فراخوانی محلی و فراخوانی راه دور

۴. نقش *Server Stub* و *Client Stub*

۵. مفهوم *Serialization* و *Deserialization*

۶. مفهوم IDL یا *Interface Definition Language*

۷. خطاهای رایج در *RPC*

۸. مقایسه *RPC* با *REST*

۹. مقایسه چند فناوری رایج *RPC*

۲-۶ فناوری هایی که باید بررسی شوند

حداقل سه مورد از فناوری های زیر باید بررسی و مقایسه شوند:

• *gRPC*

• *JSON-RPC*

• *XML-RPC*

• *Apache Thrift*

• *Java RMI*

• *SOAP*

۳-۶ محورهای مقایسه

مقایسه فناوری ها باید بر اساس معیارهای زیر انجام شود:

• نوع داده و فرمت پیام

• روش تعریف *interface*

• پشتیبانی از چند زبان برنامه نویسی

• کارایی و سربار ارتباطی

- سادگی پیاده سازی
- مناسب بودن برای سیستم های توزیع شده
- پشتیبانی از *streaming*
- موارد استفاده رایج

۴-۶ جدول مقایسه اجباری

در گزارش باید حداقل یک جدول مقایسه مانند جدول زیر وجود داشته باشد:

<i>Use Case</i>	<i>Streaming</i>	<i>IDL</i>	<i>Data Format</i>	<i>Technology</i>
<i>Microservices</i>	<i>Yes</i>	<i>Yes</i>	<i>Protocol Buffers</i>	<i>gRPC</i>
<i>Simple APIs</i>	<i>No</i>	<i>No/Optional</i>	<i>JSON</i>	<i>JSON-RPC</i>
<i>Legacy Systems</i>	<i>No</i>	<i>No</i>	<i>XML</i>	<i>XML-RPC</i>

۵-۶ پرسش های تحلیلی اجباری

در گزارش باید به پرسش های زیر پاسخ داده شود:

۱. چرا *RPC* باعث می شود فراخوانی یک سرویس راه دور شبیه فراخوانی یک تابع محلی به نظر برسد؟
۲. چرا این شباهت می تواند خطرناک یا گمراه کننده باشد؟
۳. چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارد؟
۴. در چه شرایطی *gRPC* انتخاب بهتری از *REST* است؟
۵. در چه شرایطی *REST* انتخاب ساده تر یا مناسب تر است؟

۷ بخش دوم: پیاده سازی سیستم توزیع شده چند ماشین مجازی

۱-۷ شرح کلی

در این بخش باید یک سیستم کوچک شامل چند سرویس مستقل پیاده سازی کنید. هر سرویس باید روی یک ماشین مجازی جداگانه اجرا شود و از طریق شبکه با سرویس های دیگر ارتباط برقرار کند.

۲-۷ ماشین های مجازی مورد نیاز

حداقل سه ماشین مجازی باید ساخته شود:

- *VM1*: ماشین سرویس وب
- *VM2*: ماشین سرویس احراز هویت
- *VM3*: ماشین سرویس فایل و تصاویر

در صورت نیاز می توانید یک ماشین چهارم نیز برای *Pub/Sub Broker* یا *Monitoring Subscriber* استفاده کنید، اما اجباری نیست.

۳-۷ *VM1*: سرویس وب

۱-۳-۷ وظیفه سرویس وب

این ماشین باید یک صفحه وب ساده اجرا کند. کاربر از طریق مرورگر وارد این صفحه می شود و فرم ورود را مشاهده می کند.

۲-۳-۷ نیازمندی های اجباری

۱. سرویس وب باید حداقل یک صفحه *login* داشته باشد.

۲. صفحه *login* باید شامل *username* و *password* باشد.

۳. پس از ارسال فرم، سرویس وب نباید خودش احراز هویت را انجام دهد.
۴. سرویس وب باید برای احراز هویت یک *RPC* به سرویس احراز هویت در *VM2* ارسال کند.
۵. در صورت موفق بودن ورود، صفحه خوش آمدگویی نمایش داده شود.
۶. در صورت ناموفق بودن ورود، پیام خطای مناسب نمایش داده شود.
۷. پس از ورود موفق، صفحه وب باید بتواند حداقل یک فایل یا تصویر را از *VM3* دریافت و نمایش دهد.

۳-۳-۷ رفتار مورد انتظار

نمونه رفتار سیستم:

۱. کاربر وارد آدرس سرویس وب می شود.
۲. فرم ورود نمایش داده می شود.
۳. کاربر نام کاربری و رمز عبور را وارد می کند.
۴. *VM1* درخواست احراز هویت را از طریق *RPC* به *VM2* ارسال می کند.
۵. *VM2* نتیجه را برمی گرداند.
۶. اگر نتیجه موفق بود، *VM1* صفحه اصلی را نمایش می دهد.
۷. صفحه اصلی یک تصویر یا فایل را از *VM3* دریافت می کند.

۴-۷: سرویس احراز هویت *VM2*

۱-۴-۷ وظیفه سرویس احراز هویت

این ماشین باید شامل داده کاربران و توابع مربوط به احراز هویت باشد. سرویس وب مستقیماً نباید به فایل کاربران یا داده داخلی این سرویس دسترسی داشته باشد.

۷-۴-۲ نیازمندی های اجباری

۱. سرویس احراز هویت باید یک *RPC procedure* برای بررسی ورود داشته باشد.

۲. این *procedure* باید حداقل دو ورودی داشته باشد:

● *username*

● *password*

۳. خروجی باید مشخص کند ورود موفق بوده یا خیر.

۴. داده کاربران باید در *VM2* نگهداری شود.

۵. *VM1* نباید به فایل کاربران دسترسی مستقیم داشته باشد.

۶. حداقل سه کاربر آزمایشی باید تعریف شود.

۷-۴-۳ نمونه داده کاربران

```
[
  {
    "username": "alice",
    "password": "alice123"
  },
  {
    "username": "bob",
    "password": "bob123"
  },
  {
    "username": "admin",
    "password": "admin123"
```

```
}  
]
```

نکته امنیتی: برای سادگی تمرین، نگهداری رمز عبور به صورت ساده قابل قبول است. با این حال، دانشجویانی که رمز عبور را *hash* کنند، امتیاز طراحی بهتری دریافت می کنند.

۴-۴-۷ نمونه تعریف سرویس

در صورتی که از *gRPC* استفاده می کنید، تعریف سرویس می تواند شبیه نمونه زیر باشد:

```
service AuthService {  
    rpc Login(LoginRequest) returns (LoginResponse);  
}
```

۵-۷ *VM3*: سرویس فایل و تصاویر

۱-۵-۷ وظیفه سرویس فایل

این ماشین باید نقش یک مخزن ساده فایل و تصویر را داشته باشد. سرویس وب در *VMI* باید بتواند بعد از ورود موفق، فایل یا تصویر را از این ماشین دریافت کند.

۲-۵-۷ نیازمندی های اجباری

۱. تعدادی فایل یا تصویر باید در *VM3* قرار داده شود.
۲. *VM3* باید یک سرویس ساده برای ارائه این فایل ها اجرا کند.
۳. *VMI* باید بتواند از طریق شبکه فایل یا تصویر را از *VM3* دریافت کند.
۴. مسیر یا آدرس فایل ها نباید به صورت محلی روی *VMI* باشد.
۵. در گزارش باید مشخص شود فایل ها چگونه از *VM3* دریافت شده اند.

۳-۵-۷ روش های قابل قبول

برای سرویس فایل می توانید یکی از روش های زیر را استفاده کنید:

• *HTTP File Server*

• *RPC-based File Service*

• یک سرویس ساده سفارشی با *Go*

ساده ترین روش قابل قبول استفاده از *HTTP File Server* است، اما اگر دریافت فایل را نیز با *RPC* انجام دهید، امتیاز طراحی بیشتری دریافت می کنید.

۸ بخش سوم: افزودن Pub/Sub و اعلان مصرف حافظه

۱-۸ شرح مسئله

در این بخش باید یک سازوکار ساده *Publish/Subscribe* به سیستم اضافه کنید. هدف این است که مصرف حافظه سرویس وب در *VM1* پایش شود و اگر مصرف حافظه از یک مقدار مشخص بیشتر شد، یک رخداد منتشر شود و یک *subscriber* آن را دریافت کرده و هشدار نمایش دهد.

۲-۸ نیازمندی های اجباری

۱. مصرف حافظه سرویس وب باید به صورت دوره ای پایش شود.

۲. یک آستانه مصرف حافظه تعریف شود.

۳. مقدار پیشنهادی آستانه: *300 MB*

۴. اگر مصرف حافظه از آستانه عبور کرد، یک *event* منتشر شود.

۵. حداقل یک *subscriber* باید این رخداد را دریافت کند.

۶. پس از دریافت رخداد، باید یک *alert* واضح نمایش داده شود.

۷. باید راهی برای افزایش عمدی مصرف حافظه سرویس وب فراهم شود تا هشدار قابل تست باشد.

۳-۸ رخداد مورد انتظار

رخداد منتشر شده می تواند شامل اطلاعات زیر باشد:

```
{  
  "event_type": "HIGH_MEMORY_USAGE",  
  "service": "web-server",  
  "memory_mb": 345,  
  "threshold_mb": 300,  
  "timestamp": "2026-..."  
}
```

۴-۸ روش های قابل قبول برای Pub/Sub

یکی از روش های زیر قابل قبول است:

- پیاده سازی ساده *Pub/Sub* با *TCP* یا *HTTP*

- استفاده از *Redis Pub/Sub*

- استفاده از *RabbitMQ*

- استفاده از *NATS*

- پیاده سازی ساده با *WebSocket*

برای ساده سازی، پیاده سازی یک *broker* کوچک با *Go* قابل قبول است. در این حالت، سرویس وب نقش *publisher* را دارد و یک برنامه دیگر نقش *subscriber* را ایفا می کند.

۵-۸ پایش مصرف حافظه

برای پایش مصرف حافظه می توانید از امکانات استاندارد *Go* استفاده کنید. به عنوان نمونه، استفاده از *runtime.ReadMemStats* قابل قبول است. متریک پیشنهادی:

• *Alloc*

• *HeapAlloc*

• *Sys*

۶-۸ افزایش عمدی مصرف حافظه

برای تست سیستم، باید یک روش مشخص برای افزایش مصرف حافظه سرویس وب پیاده سازی کنید. برای مثال:

• یک *endpoint* به نام */consume-memory*

• با هر بار فراخوانی این *endpoint* مقداری حافظه در برنامه تخصیص داده شود

• حافظه تخصیص داده شده باید در یک ساختار سراسری نگهداری شود تا بلافاصله آزاد نشود

نمونه فراخوانی:

```
curl http://WEB_VM_IP:8080/consume-memory?mb=100
```

۷-۸ رفتار مورد انتظار

۱. سرویس وب اجرا می شود.

۲. *subscriber* اجرا می شود و منتظر رخداد می ماند.

۳. مصرف حافظه سرویس وب کمتر از آستانه است.

۴. با فراخوانی *consume-memory* مصرف حافظه افزایش پیدا می کند.

۵. پس از عبور از آستانه، سرویس وب یک رخداد منتشر می کند.

۶. *subscriber* رخداد را دریافت می کند.

۷. پیام هشدار نمایش داده می شود.

۸-۸ پروتکل ها و فناوری های مجاز

برای ارتباط *RPC* بین *VM1* و *VM2* می توانید از یکی از روش های زیر استفاده کنید:

- *gRPC*

- *JSON-RPC*

- *XML-RPC*

- پیاده سازی ساده *RPC-like* با *HTTP* و *JSON*

با این حال، اگر از *HTTP REST* ساده استفاده می کنید، باید در گزارش توضیح دهید چرا آن را *RPC-like* در نظر گرفته اید و تفاوت آن با *RPC* واقعی چیست. استفاده از *gRPC* یا *JSON-RPC* امتیاز طراحی بیشتری دارد.

۹-۸ سناریوی تست اجباری

در گزارش باید سناریوی تست زیر را اجرا و مستند کنید:

۱. نمایش آدرس و *IP* هر ماشین مجازی

۲. اجرای سرویس احراز هویت در *VM2*

۳. اجرای سرویس فایل در *VM3*

۴. اجرای سرویس وب در *VMI*
۵. باز کردن صفحه وب از سیستم میزبان یا یک کلاینت دیگر
۶. تلاش برای ورود با اطلاعات نادرست
۷. مشاهده پیام خطای ورود
۸. تلاش برای ورود با اطلاعات درست
۹. مشاهده صفحه موفقیت
۱۰. دریافت یا نمایش فایل یا تصویر از *VM3*
۱۱. اجرای *subscriber*
۱۲. افزایش عمدی مصرف حافظه سرویس وب
۱۳. مشاهده هشدار مصرف حافظه بالا

۹ مستندات و گزارش

گزارش نهایی باید شامل موارد زیر باشد:

- معماری کلی سیستم
- نقش هر ماشین مجازی
- آدرس *IP* هر ماشین مجازی
- فناوری انتخاب شده برای *RPC*
- دلیل انتخاب فناوری *RPC*
- توضیح *procedure* های پیاده سازی شده

- توضیح نحوه دریافت فایل از *VM3*
- توضیح سازوکار *Pub/Sub*
- توضیح روش پایش حافظه
- اسکرین شات یا خروجی ترمینال از اجرای موفق هر بخش
- مشکلات و محدودیت های پیاده سازی

۱۰ موارد تحویلی

موارد زیر باید تحویل داده شوند:

- کد کامل سرویس وب
- کد کامل سرویس احراز هویت
- کد کامل سرویس فایل
- کد مربوط به *Pub/Sub*
- فایل داده کاربران
- فایل های نمونه یا تصاویر مورد استفاده
- گزارش نهایی به صورت *PDF*
- فایل **README** برای هر سرویس
- اسکرین شات های اجرای سیستم

۱۱ معیار ارزیابی

۱-۱۱ بخش اول: گزارش، RPC 25 نمره

- توضیح دقیق مفهوم *RPC*: 5
- توضیح اجزای *RPC* مانند *stub*، *serialization*، و *IDL*: 5
- مقایسه حداقل سه فناوری *RPC*: 6
- مقایسه *RPC* و *REST*: 4
- پاسخ به پرسش های تحلیلی: 5

۲-۱۱ بخش دوم: سیستم چند ماشین مجازی، 50 نمره

- راه اندازی صحیح حداقل سه ماشین مجازی: 8
- پیاده سازی صفحه وب و فرم ورود: 7
- پیاده سازی صحیح سرویس احراز هویت: 8
- استفاده واقعی از *RPC* بین *VM1* و *VM2*: 10
- جداسازی داده کاربران از سرویس وب: 4
- پیاده سازی سرویس فایل در *VM3*: 6
- دریافت موفق فایل یا تصویر از *VM3*: 4
- مدیریت خطا و کیفیت کد: 3

۱۱-۳ بخش سوم: Pub/Sub و پایش حافظه، 25 نمره

- پایش دوره ای مصرف حافظه سرویس وب: 5
- تعریف و بررسی آستانه مصرف حافظه: 4
- پیاده سازی انتشار رخداد: 5
- پیاده سازی *subscriber* و دریافت رخداد: 5
- فراهم کردن روش افزایش عمدی مصرف حافظه: 3
- مستندسازی و نمایش هشدار: 3

۱۲ موارد کسر نمره

موارد زیر باعث کسر نمره خواهند شد:

- اجرای همه سرویس ها روی یک ماشین بدون استفاده واقعی از چند *VM*
- استفاده از *localhost* به جای آدرس شبکه ماشین های مجازی
- انجام احراز هویت در سرویس وب به جای *VM2*
- نبودن فایل کاربران در *VM2*
- نبودن ارتباط واقعی بین *VM1* و *VM3*
- نبودن *Pub/Sub* واقعی و صرفا چاپ دستی هشدار
- نبودن سناریوی تست قابل بازتولید
- نبودن *README*
- نبودن اسکرین شات یا خروجی ترمینال برای اثبات اجرا

۱۳ دستاوردهای تمرین

در پایان این تمرین باید بتوانید نشان دهید که:

- مفهوم *RPC* و فناوری های مختلف آن را می شناسید
- می توانید چند سرویس مستقل را روی چند ماشین مجازی اجرا کنید
- می توانید یک سرویس وب را به یک سرویس احراز هویت راه دور متصل کنید
- می توانید یک سرویس فایل جداگانه طراحی کنید
- می توانید رخدادهای سیستمی را با الگوی *Publish/Subscribe* منتشر و دریافت کنید
- می توانید رفتار یک سیستم توزیع شده را مستند و تحلیل کنید